Strategy Evaluation: Manual policy vs. Q-Learning

Erich Sung
esung39@gatech.edu

***Abstract*—**In this paper, we build and discuss two AI-based trading models: one that operates based on a manual policy, referred to as the Manual Strategy, and another that uses Q-learning to learn policies themselves, referred to as the Strategy Learner. In both models, technical indicators are used to predict future market movements and determine the best course of action based on those predictions. In the first experiment, we compare and study the portfolio values generated by both models. For the second experiment, we vary market impact in the strategy learner and observe how it changes the outcome.

1 INTRODUCTION

Our goal is to build and test an autonomous trading agent that makes trading decisions based on technical indicators. The model will generate trading signals based on a manual policy or a policy learned through reinforcement learning, specifically Q-learning. These signals are represented as a DataFrame and computed in a market simulator for testing. By comparing the results of the two cases, we aim to study the strengths and effectiveness of each approach.

Our initial hypothesis is that while a reinforcement learning-based model can produce more optimized results for in-sample data, it may be prone to overfitting. Given relatively short training data, it may not fully capture policies that can be applied to the real world, especially if the indicators are not carefully chosen. Therefore, out-of-sample testing for the reinforcement learning model may perform poorly, especially if the parameters are not carefully chosen.

2 INDICATOR OVERVIEW

2.1 Bollinger Bands

Many indicators require calculating a Simple Moving Average (SMA) before calculating their actual values. The Bollinger Bands are no exception. SMA can be easily calculated by applying the rolling method to the data over the desired lookback window and taking the mean. The standard deviation (σ) can also be

computed over the same rolling window. We can then calculate the Bollinger bands as (Sung, 2026):

$$\begin{aligned} \text{Upper Band} &= \text{SMA} + 2\sigma \\ \text{Lower Band} &= \text{SMA} - 2\sigma \end{aligned} \tag{1}$$

To use the indicator in a systematic model, it is important to develop a single parameter that contains information about both bands. One such parameter is %B. The following equation can be used to compute %B (Sung, 2026).

$$\%B = \frac{\text{Price} - \text{Lower Band}}{\text{Upper Band} - \text{Lower Band}} \tag{2}$$

%B tells us where the price is relative to the two bands. If %B is 1 or greater, the price is at or above the upper band. If %B is 0 or lower, then the price is at or below the lower band.

2.2 Stochastic Oscillator

Stochastic Oscillator consists of two values: %K and %D. First, %K is calculated using the following equation (Sung, 2026):

$$\%K_t = \frac{C_t - L_{t,n}}{H_{t,n} - L_{t,n}} * 100 \tag{3}$$

where n is the lookback window (typically 14 days), C_t si the current closing price, $L_{t,n}$ is the lowest price over the previous n days, and $H_{t,n}$ is the highest price over the previous n days.

%D is the three-day average of %K values, calculated as follows (Sung, 2026):

$$\%D_t = \frac{\%K_t + \%K_{t-n} + \%K_{t-2n}}{3} \tag{4}$$

We are interested in where %K and %B intersect, and we need to compute a parameter that ties the two values. However, either %K or %B can be zero, meaning that taking their ratio can lead to division by zero. To avoid this issue, we can compute their difference instead:

$$SO = \%K - \%D \tag{5}$$

2.3 Momentum

Momentum is one of the more fundamental indicators. To compute momentum, simply divide the price on the target date by the price n days earlier (Sung, 2026):

$$\text{Momentum[t]} = \frac{\text{Price[t]}}{\text{Price[t-n]}} - 1 \tag{6}$$

In this equation, -1 simply shifts the value so that the price difference can be read more naturally. For example, a momentum value of 0.1 would indicate that the price has risen by 10% over the given lookback window.

Unlike the previous indicators, momentum does not require any additional parameter to be sued by our trading agent, as its value directly provides sufficient information on whether the price is rising or falling.

3 MANUAL STRATEGY

For the Manual Strategy model, we provide a specific set of rules that are used to make trading decisions based on the three indicators discussed in the previous sections. Therefore, we first need to examine how each indicator can be used to generate buy or sell signals.

%B crosses provide overbought or oversold signals. If the value crosses below 1 from a higher value, we expect the price to decline, indicating a sell signal. Conversely, if %B crosses above 0 from a lower value, it signals a buy. The Stochastic Oscillator also provides information on overbought and oversold levels. If the %K-%D value approaches 0 from a higher value, then we sell, and if the value approaches 0 from a lower value, then we buy. Lastly, momentum indicates if the price is currently rising or falling. Crossing 0 from a higher value signals the start of a decline in price, which indicates a sell signal, while crossing 0 from a lower value signals the start of a rise in price, indicating a buy signal (Sung, 2026).

With these rules, we use each indicator to create a table of target positions identified by -1 and +1, which represent short and long, respectively. One thing to note is that we only allow short or long positions, but not a no-position state. The reason for this is twofold: 1. We want to simplify computation of signals. Assigning different rules for exit conditions may overly complicate the model, making it less robust. 2. It generalizes better for our specific strategy, as we try to combine the three indicators to generate comprehensive signals. This will be discussed further in the following paragraph, as we examine how the signals are combined.

Now that we have a strategy for each individual indicator, the next step is to devise a method that combines the results of all the indicators. One such method is to simply add the signals across all the indicators to identify "strong" signals. If

all indicators detect buy or sell signals at a specific time, the combined value will be either -3 or +3, and these are considered "strong" signals. Any values between -3 and +3 indicate mixed signals, which are considered "weak". To translate these results into a vector, we replace -3 and +3 with -1 and +1, respectively, and set all other values to 0. However, the resulting vector cannot be directly used to buy when the value is +1 and sell when the value is -1, because it only signifies when it is “advantageous” to buy or sell, and may contain repeated buy or sell signals, which is not allowed. Therefore, the target position vector first needs to be computed by using a fill forward. This ensures that the strategy maintains its current position until a strong signal indicates that the position should be reversed, so that the model adheres to our rule.

The idea behind this method is that the Stochastic Oscillator and %B indicate whether the asset is overbought or oversold, while momentum describes the market trend. The combination of the oscillator and %B can provide information about potential price reversals, while momentum can confirm if the current trend is consistent with the expected market shift. Additionally, we wanted to be conservative in our approach, so trades only occur if all three indicators agree. Once we have a target table, it is easy to create a trade table containing the actions that need to be taken to reach the target by shifting the DataFrame and subtracting it from the original target table.

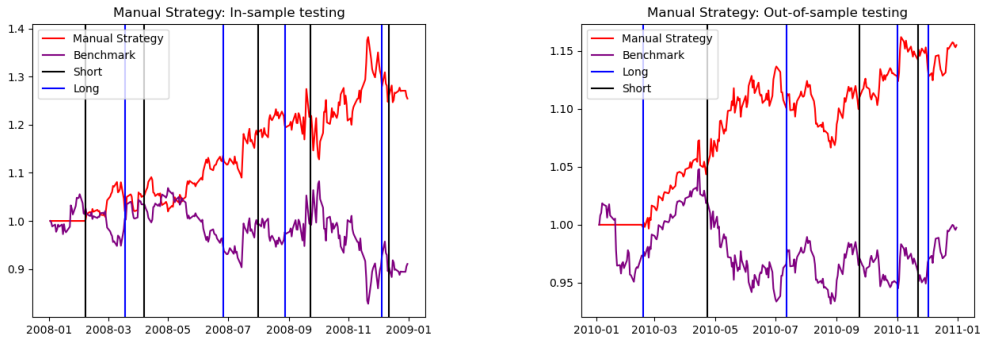


Figure 1—Manual Strategy: Normalized portfolio values for in-sample and out-of-sample testing. The Sharpe Ratio was 1.068 for in-sample testing and 1.521 for out-of-sample testing.

Figure 1 shows the normalized portfolio values for in-sample and out-of-sample testing of the Manual Strategy compared with the benchmark, which represents the expected return from buying and holding 1,000 shares without any further

trades. We can see that the in-sample strategy generated a greater cumulative return. One interesting observation is that out-of-sample testing produced a higher Sharpe ratio despite a lower cumulative return, indicating better risk-adjusted performance. Detailed statistics are shown in Table 1.

One explanation for the higher cumulative return in in-sample testing is the presence of parameter tuning. We tested different lookback windows for each indicator and chose values that generated the highest cumulative return on the in-sample data. Another contributing factor may be the nature of the data itself. We can observe that in-sample testing had larger volatility. This means there were more opportunities to exploit by using technical analysis. This reflects the general principle in finance that higher potential returns are typically associated with greater risk. Regardless, the higher Sharpe Ratio observed in the out-of-sample testing suggests that the Manual Strategy was able to generalize reasonably well to unseen data.

4 STRATEGY LEARNER

In contrast to creating a manual strategy and adhering to its buy and sell signals, the Strategy Learner aims to learn policies from the data itself using reinforcement learning, specifically Q-Learning. To build a Q-Learning model, we need to identify three elements: state, action, and reward. In a general sense, the state includes features about the asset: in our case, indicators and current position. The action is simply buy, sell, or do nothing. Finally, the reward is the profit or loss we incur by taking an action in the market.

The model works by updating the Q-table. Commonly represented as Q[s,a], it stores the value of taking action a in state s, based on the immediate reward and expected future discounted rewards. The new state, Q'[s,a], can be represented as (Balch, 2026):

$$Q'[s,a] = (1-\alpha)Q[s,a] + \alpha * (r + \gamma * Q[s', argmax'_a(Q[s',a'])] \quad (7)$$

where s represents the current state, a represents the action, and r represents the reward.α represents the learning rate, and γ represents the future discount reward; both will be discussed in more detail later. Once learning is complete, usually using historical price data, the most rewarding action can be queried by providing the current state based on the same indicators used during training.

To have an effective Q-learner, it is important to understand each hyperparameter

and set them appropriately. The learning rate, α, determines how quickly new experiences update the Q-values. This was set to 0.1 to allow more gradual learning. γ represents the discount rate, determining how much we value later rewards. This was set to 0.8 so that we can prioritize future rewards while still emphasizing immediate rewards. rar, or random action rate, was chosen to be 0.8 to allow more exploration. radr is the rate at which rar decreases, and was set to 0.99998. This was set relative to the total possible iterations, so that the model still has a chance to explore even at the later phases of the training. dyna is not used for the model and was set to 0.

One important step in using Q-Learning is discretizing features representing the current state into a single integer value. The process involves first normalizing each indicator using the following equation (Balch, 2026):

$$\text{Normalized_Indicator[t]} = \frac{\text{Indicator[t] - Mean}}{\text{Indicator.std()}} \tag{8}$$

This allows indicator values to be scaled more evenly around 0. One important step is to save the normalized factors, namely the means and the standard deviations, so that they can later be reused to normalize test indicator values. The next step is to set up bins that these values belong to. While this can be done using Pandas methods, such as qcut(), our model uses manually defined boundaries [-∞, -1, -0.25, 0, 0.25, 1, +∞] to simplify the process. An integer is assigned to each bin, so an indicator value falling within a specific bin is assigned the corresponding integer, which is then used to compute the current state as:

$$\begin{aligned}\text{State} = (3 * 6^2 * \text{Indicator_1}) + (3 * 6 * \text{Indicator_2}) \\ + (3 * \text{Indicator_3}) + \text{Current Position}\end{aligned} \tag{9}$$

Here, 3 is the total number of possible current positions, and 6 is the total number of bins. To illustrate, if our indicators are discretized to [0,3,5] and our current position is long, represented as 2, our discretized state would then be $(3 * 6^2 * 0) + (3 * 6 * 3) + (3 * 5) + 2 = 65$. Therefore, in the Q-Table, 65 represents a specific state and would be paired with the possible actions and their corresponding Q-values. Using these values, the total number of possible states can also be calculated as 6*6*6*3=648.

5 EXPERIMENT 1

For this experiment, we compare the Manual Strategy and the Strategy Learner. Our hypothesis is that the Manual Strategy will show similar performance be-

tween in-sample and out-of-sample data, while the Strategy Learner will perform significantly better in in-sample testing than in out-of-sample testing. This is because the strategy used in the Manual Strategy model was derived from general trading heuristics, while Q-Learning learns the patterns from the training data and is therefore more prone to overfitting.

To ensure both models are tested under the same realistic conditions, a few assumptions need to be made. We assume that trades are executed the day after the signal, since indicator values cannot be computed until the market is closed for the day. The allowed positions are -1000 (short), 0 (no position), or +1000 (long) shares. Only regular buy or sell actions are allowed. This means stops, trailing stops, stop-loss, or any other trading setup are not allowed. The commission is $9.95 per trade, and the market impact is 0.005 per share. For Bollinger Bands, we use a lookback window of 10. For the Stochastic oscillator, we use a lookback window of 15. For momentum, we use a lookback window of 10. For in-sample testing, we use 'JPM' prices from 01/01/2008 to 01/01/2009. For out-of-sample testing, we use 'JPM' prices from 01/01/2010 to 01/01/2011. Lastly, a random seed is used to ensure that the same results are achieved each time the program is executed. This helps ensure consistent results for analysis in the paper.

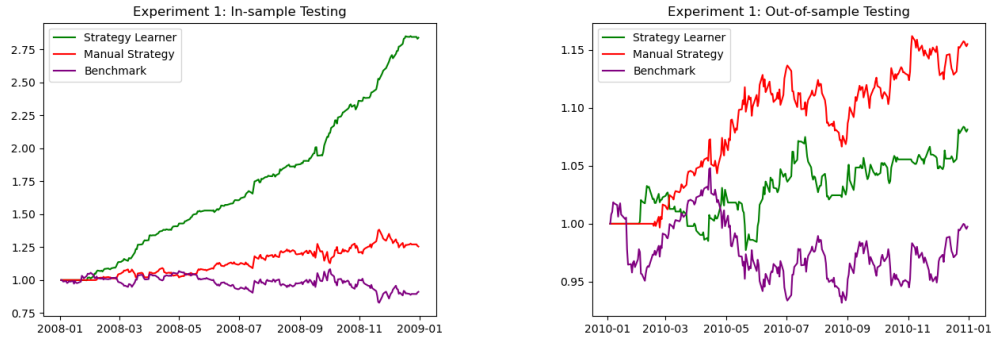


Figure 2—Normalized portfolio values for the Manual Strategy model and the Strategy Learner model. The left plot represents in-sample testing results, and the right plot represents out-of-sample testing results.

Figure 2 represents the portfolio value of each model, along with the benchmark, and Table 1 shows performance statistics for Experiment 1. In all cases, our models outperformed the benchmark to varying degrees. As already discussed

Table 1—Experiment1: Performance statistics for the benchmark, Manual Strategy, and Strategy Learner.

Category:	Benchmark	Manual Strategy	Strategy Learner
In Sample Data			
Cumulative return	-0.089363	0.254226	1.837281
Average Daily return	-0.000192	0.001012	0.004179
Standard Deviation	0.019011	0.015043	0.008007
Sharpe Ratio	-0.160109	1.067559	8.284610
Out of Sample Data			
Cumulative return	-0.002505	0.154943	0.081287
Average Daily return	0.000019	0.000593	0.000328
Standard Deviation	0.007601	0.006192	0.005798
Sharpe Ratio	0.039253	1.520764	0.898403

in the Manual Strategy section, the model produced higher cumulative returns for in-sample data, but the out-of-sample testing resulted in a higher Sharpe ratio.

For the Strategy Learner model, its in-sample performance significantly outperformed all other cases, including its own out-of-sample performance. One likely explanation for this is the different pattern between the data sets. From February of 2010 to April of 2010, the out-of-sample data shows a steep rise in price, followed by a sharp decline over the next few months. Because such a pattern was not present in the in-sample data, the learner likely was not able to learn the pattern to capitalize on the price movement.

The result supports our initial hypothesis. The performance of the Manual Strategy did not vary much across the datasets. In contrast, the Strategy Learner performed significantly better on the in-sample data. We would expect this relative behavior most of the time with in-sample data, meaning that the Strategy Learner will likely outperform the Manual Strategy on the training data. The Q-Learner needs to learn policies based on the data it is trained on. If it has learned any meaningful patterns from the training data, its performance would also be optimized for that specific data, which may not generalize to the outside world. The most effective way to remedy this would be to provide a larger and more

diverse dataset and find optimal parameters so that the learner can effectively handle unexpected market conditions.

6 EXPERIMENT 2

In experiment 2, we set market impact to 0, 0.005, and 0.05 and observe how varying impact affects the Strategy Learner. The commission is set to $0 for this experiment, and all other testing conditions remain the same as in Experiment 1. We hypothesize that an increase in impact will result in lower performance and cause the model to take on a more conservative trading strategy.

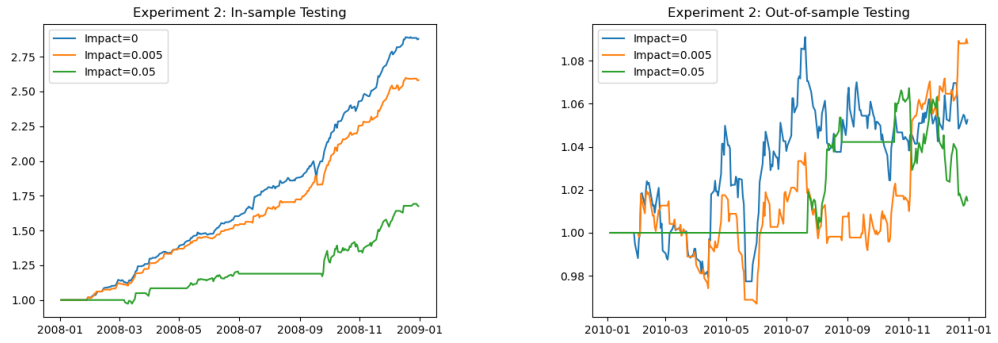


Figure 3—Normalized portfolio values for the Strategy Learner model with different market impact. The left plot represents in-sample testing results, and the right plot represents out-of-sample testing results.

Figure 3 shows portfolio values for applying different impact values, and Table 2 presents statistics for each case. The cumulative returns do not show any linear relationship with impact, as the highest cumulative return was achieved when the impact was 0.005 for out-of-sample testing. This shows that even if there is a small penalty for taking actions, the potential reward from making profitable trades can still outweigh the costs. Because each test case was conducted using the same random seed, running the test with another seed may yield different results and affect the validity of the test. If this project were to be conducted again, We could mitigate this issue by running each test multiple times with different seeds and averaging the results.

By evaluating the plots for in-sample testing with 0 and 0.005 impact, we can tell that they have similar trends. From this observation, we can infer that the trading behavior did not change drastically. However, when the impact is increased to

Table 2—Experiment2: Performance statistics for the Strategy Learner with varying market impact.

Impact:	**0**	**0.005**	**0.05**
In Sample Data			
Cumulative return	1.878400	1.580935	0.675330
Average Daily return	0.004238	0.003802	0.002115
Standard Deviation	0.008227	0.008140	0.011557
Sharpe Ratio	8.177201	7.415449	2.905622
Out of Sample Data			
Cumulative return	0.052500	0.088160	0.014955
Average Daily return	0.000222	0.000350	0.000066
Standard Deviation	0.006094	0.005267	0.003576
Sharpe Ratio	0.579194	1.056232	0.290793

0.05, the patterns in the plot change significantly. The model takes visibly fewer actions. What is interesting is that it often stays in a 'cash out' position, where it takes neither a long nor a short position, as evidenced by the frequent flat lines. One possible explanation is that the model learned that staying out is more beneficial, regardless of the current position. Therefore, it tries to exit its position more frequently, even if there is a large cost associated with it.

Another interesting piece of data to examine is the standard deviations for in-sample data. While the high-impact plot (0.05) has more frequent flat lines due to fewer trades and longer cash-out positions, it has a higher standard deviation compared to other test cases. This may seem counterintuitive, as higher noise in the plot is often associated with higher standard deviation. From this observation, we can conclude that simply having fewer actions does not necessarily guarantee lower risk; rather, it depends on the quality of the action taken. In conclusion, the test results do not fully support the initial hypothesis. While an increase in market impact resulted in lower performance at an extremely high value, the relationship was not linear. Furthermore, trading behavior exhibited substantially different patterns only for the high-impact test case.

7 REFERENCES

[1] Balch, T. R. (2026a). "Q-Learning: Update Rule [Video lecture]". In: *CS7646: Machine Learning for Trading*. Georgia Institute of Technology.

[2] Balch, T. R. (2026b). "Technical Analysis: Normalization [Video lecture]". In: *CS7646: Machine Learning for Trading*. Georgia Institute of Technology.

[3] Sung, E. (2026). "Developing Systematic Methods for Identifying Buy and Sell Signals Using Multiple Technical Indicators". In.